Striver SDE Sheet — Arrays Part III

---

1. Search in a 2D Matrix

Problem

Given an `m × n` matrix where:

- Every row is sorted.
- The first element of each row is greater than the last element of the previous row.

Find whether a target exists.

Example:

```
1   3   5   7
10  11  16  20
23  30  34  60
```

Target:

```
16
```

Output:

```
true
```

Approach 1: Brute Force

Traverse every element and compare it with the target.

```
for each row:
    for each element:
        if element == target:
            return true
```

Complexity

- Time: `O(m × n)`
- Space: `O(1)`

Approach 2: Better — Binary search each row

Because every row is sorted:

1. Check whether the target can lie in the current row.
2. Apply binary search on that row.

A target can lie in row `i` when:

```
matrix[i][0] <= target <= matrix[i][n - 1]
```

Complexity

- Finding the relevant row: `O(m)`
- Binary search: `O(log n)`
- Total: `O(m + log n)`
- Space: `O(1)`

Approach 3: Optimal — Treat matrix as a 1D sorted array

Core idea

The complete matrix behaves like one sorted array of size:

```
m × n
```

Use binary search on virtual indices:

```
0 to (m × n) - 1
```

Convert a virtual index `mid` into matrix coordinates:

```
row = mid / n
column = mid % n
```

Therefore:

```
matrix[row][column]
=
matrix[mid / n][mid % n]
```

Algorithm

```
low = 0
high = m × n - 1

while low <= high:

    mid = low + (high - low) / 2

    row = mid / n
    column = mid % n

    value = matrix[row][column]

    if value == target:
        return true

    else if value < target:
        low = mid + 1

    else:
        high = mid - 1

return false
```

Why does index conversion work?

For a matrix with `n` columns:

```
virtual index = row × n + column
```

Reversing it:

```
row = index / n
column = index % n
```

Complexity

- Time: `O(log(m × n))`
- Space: `O(1)`

Revision key

Flatten the matrix logically, not physically. Use `mid / columns` and `mid % columns`.

---

2. Pow(x, n)

Problem

Calculate:

```
xⁿ
```

Example:

```
x = 2
n = 10
```

Output:

```
1024
```

The exponent may also be negative.

Example:

```
2⁻² = 1 / 2² = 0.25
```

Approach 1: Brute Force

Multiply `x` by itself `n` times.

```
answer = 1

for i from 1 to n:
    answer *= x
```

For a negative exponent:

```
answer = 1 / answer
```

Complexity

- Time: `O(|n|)`
- Space: `O(1)`

Approach 2: Optimal — Binary Exponentiation

Core idea

Instead of multiplying `x` repeatedly, reduce the exponent by half.

For an even exponent:

```
xⁿ = (x²)ⁿ/²
```

For an odd exponent:

```
xⁿ = x × xⁿ⁻¹
```

Iterative algorithm

```
answer = 1
power = n
```

If `power` is negative:

```
power = -power
```

Then:

```
while power > 0:

    if power is odd:
        answer = answer × x
        power = power - 1

    else:
        x = x × x
        power = power / 2
```

At the end:

```
if n < 0:
    return 1 / answer

return answer
```

More concise version

```
while power > 0:

    if power % 2 == 1:
        answer *= x

    x *= x
    power /= 2
```

The odd exponent is handled automatically through integer division.

Example

Calculate:

```
2¹⁰
```

```
power = 10, x = 2, answer = 1

10 even:
x = 4, power = 5

5 odd:
answer = 4
x = 16, power = 2

2 even:
x = 256, power = 1

1 odd:
answer = 4 × 256 = 1024
```

Important edge case: `INT_MIN`

For a 32-bit integer:

```
INT_MIN = -2147483648
```

Its positive value cannot fit inside an `int`.

Therefore, store the exponent in a 64-bit integer:

```
long long power = n
```

Then safely perform:

```
power = -power
```

Complexity

- Time: `O(log |n|)`
- Space: `O(1)`

Revision key

Odd exponent: multiply answer. Even exponent: square the base and halve the power.

---

3. Majority Element — More Than n/2 Times

Problem

Find the element that appears more than:

```
n / 2
```

times.

It is assumed that a majority element exists.

Example:

```
[2,2,1,1,1,2,2]
```

Output:

```
2
```

Approach 1: Brute Force

For every element, count its frequency.

```
for each element:
    count = 0

    for each value:
        if value == element:
            count++

    if count > n / 2:
        return element
```

Complexity

- Time: `O(n²)`
- Space: `O(1)`

Approach 2: Better — Hash map

Store the frequency of every element.

```
frequency[value]++
```

Return an element when:

```
frequency[value] > n / 2
```

Complexity

- Time: `O(n)` average
- Space: `O(n)`

Approach 3: Better — Sorting

After sorting, the majority element must occupy the middle index:

```
arr[n / 2]
```

Why?

An element occurring more than half the array cannot avoid covering the middle position after sorting.

Complexity

- Time: `O(n log n)`
- Space: depends on sorting

Approach 4: Optimal — Moore's Voting Algorithm

Core idea

The majority element appears more times than all other elements combined.

Cancel every occurrence of the candidate with a different element.

The majority element remains at the end.

Phase 1: Find a candidate

Maintain:

```
candidate
count
```

Algorithm:

```
count = 0

for value in array:

    if count == 0:
        candidate = value
        count = 1

    else if value == candidate:
        count++

    else:
        count--
```

At the end, `candidate` is the possible majority element.

Phase 2: Verify the candidate

If the problem does not guarantee that a majority element exists:

```
count occurrences of candidate

if count > n / 2:
    return candidate

return -1
```

Example

```
[2,2,1,1,1,2,2]
```

```
Value   Candidate   Count

2       2           1
2       2           2
1       2           1
1       2           0
1       1           1
2       1           0
2       2           1
```

Candidate:

```
2
```

Complexity

- Candidate selection: `O(n)`
- Verification: `O(n)`
- Total: `O(n)`
- Space: `O(1)`

Revision key

> Equal element increases the vote; different element cancels one vote.

---

4. Majority Element II — More Than n/3 Times

Problem

Find all elements appearing more than:

```
n / 3
```

times.

Example:

```
[1,1,1,3,3,2,2,2]
```

Output:

```
[1,2]
```

Important observation

At most two elements can appear more than `n/3` times.

Why?

If three different elements appeared more than `n/3` times:

```
frequency1 + frequency2 + frequency3 > n
```

This is impossible.

Therefore, we only need two candidates.

Approach 1: Brute Force

For each unique element:

1. Count its occurrences.

2. Add it if the count exceeds `n/3`.

Avoid inserting the same element repeatedly.

Complexity

- Time: `O(n²)`
- Space: up to `O(1)` excluding output because the answer has at most two elements

---

Approach 2: Better — Hash map

Store frequencies.

```
frequency[value]++
```

Add an element when:

```
frequency[value] > n / 3
```

Complexity

- Time: `O(n)` average
- Space: `O(n)`

Approach 3: Optimal — Extended Moore's Voting Algorithm

Maintain two candidates:

```
candidate1, candidate2
count1, count2
```

Phase 1: Select possible candidates

The order of conditions matters.

```
count1 = 0
count2 = 0
```

```
for value in array:

    if count1 == 0 and value != candidate2:
        candidate1 = value
        count1 = 1

    else if count2 == 0 and value != candidate1:
        candidate2 = value
        count2 = 1

    else if value == candidate1:
        count1++

    else if value == candidate2:
        count2++

    else:
        count1--
        count2--
```

An alternative valid ordering checks candidate equality before checking empty counts.

Phase 2: Verify both candidates

Reset counts:

```
count1 = 0
count2 = 0
```

Count their actual frequencies.

```
for value in array:

    if value == candidate1:
        count1++

    else if value == candidate2:
        count2++
```

Add candidates satisfying:

```
count > n / 3
```

Why do we need verification?

Moore's Voting only provides possible candidates.

An array may contain no element occurring more than `n/3` times.

Example:

```
[1,2,3,4]
```

General rule

For elements appearing more than:

```
n / k
```

there can be at most:

```
k - 1
```

such elements.

Complexity

- Candidate selection: `O(n)`
- Verification: `O(n)`
- Total: `O(n)`
- Space: `O(1)` excluding output

Revision key

For `n/3`, maintain two candidates because there can be at most two valid answers.

---

5. Grid Unique Paths

Problem

A robot starts at:

```
(0, 0)
```

and must reach:

```
(m - 1, n - 1)
```

It can move only:

```
right or down
```

Find the number of unique paths.

Example:

```
m = 3, n = 3
```

Output:

```
6
```

Approach 1: Brute Force — Recursion

At every cell, try:

```
move right
move down
```

Recursive relation

```
paths(i, j)
=
paths(i + 1, j)
+
paths(i, j + 1)
```

Base cases:

```
if i == m - 1 and j == n - 1:
    return 1

if i >= m or j >= n:
    return 0
```

Complexity

- Time: approximately `O(2^(m+n))`
- Space: `O(m+n)` recursion stack

This recalculates the same states many times.

Approach 2: Better — Dynamic Programming

Memoization

Store the answer for every state:

```
dp[i][j]
```

Before solving a state:

```
if dp[i][j] is already calculated:
    return dp[i][j]
```

Complexity

- Time: `O(m × n)`
- Space: `O(m × n)` plus recursion stack

Tabulation

Create:

```
dp[m][n]
```

Initialize the starting cell:

```
dp[0][0] = 1
```

For every cell:

```
fromTop = dp[i - 1][j]
fromLeft = dp[i][j - 1]

dp[i][j] = fromTop + fromLeft
```

Check boundaries before accessing top or left.

Complexity

- Time: `O(m × n)`
- Space: `O(m × n)`

Space-optimized DP

Only the previous row is needed.

Maintain:

```
previous[n]
current[n]
```

Or use a single array.

Complexity

- Time: `O(m × n)`
- Space: `O(n)`

Approach 3: Optimal — Combinations

Core observation

To reach the destination, the robot must perform exactly:

```
m - 1 downward moves
n - 1 rightward moves
```

Total moves:

```
m + n - 2
```

We only need to choose where the downward moves occur:

```
Answer = C(m + n - 2, m - 1)
```

Or choose right moves:

```
Answer = C(m + n - 2, n - 1)
```

Both are equal.

Efficient combination calculation

Let:

```
N = m + n - 2
r = min(m - 1, n - 1)
```

Then:

```
answer = 1

for i from 1 to r:
    answer = answer × (N - r + i)
    answer = answer / i
```

Example

For:

```
m = 3
n = 3
```

Total moves:

```
4
```

Choose two downward moves:

```
C(4,2) = 6
```

Complexity

- Time: `O(min(m,n))`
- Space: `O(1)`

Revision key

> Every path contains fixed right and down moves. Count their different arrangements using combinations.

---

6. Reverse Pairs

Problem

Count pairs `(i, j)` such that:

```
i < j
```

and:

```
arr[i] > 2 × arr[j]
```

Example:

```
[1,3,2,3,1]
```

Reverse pairs:

```
(3,1)
(3,1)
```

Answer:

```
2
```

Difference from inversion count

An inversion requires:

```
arr[i] > arr[j]
```

A reverse pair requires:

```
arr[i] > 2 × arr[j]
```

Both can be solved using modified merge sort, but the counting logic differs.

Approach 1: Brute Force

Check every pair:

```
count = 0

for i from 0 to n - 1:
    for j from i + 1 to n - 1:

        if arr[i] > 2 × arr[j]:
            count++
```

Complexity

- Time: `O(n²)`
- Space: `O(1)`

Approach 2: Optimal — Modified Merge Sort

Core idea

Divide the array into two sorted halves.

Before merging them, count valid cross-pairs where:

```
i belongs to the left half
j belongs to the right half
```

Because both halves are sorted, use two pointers.

Counting cross-pairs

For every element in the left half:

```
for i from low to mid:
```

Move `right` while:

```
arr[i] > 2 × arr[right]
```

```
right = mid + 1

for i from low to mid:

    while right <= high
          and arr[i] > 2LL × arr[right]:

        right++

    count += right - (mid + 1)
```

Why does the right pointer not reset?

The left half is sorted.

As `arr[i]` becomes larger, every right-side value valid for the previous left element remains valid.

Therefore, `right` only moves forward.

Then perform normal merge

After counting pairs, merge the two sorted halves as in merge sort.

```
mergeSort(low, high):

    if low >= high:
        return 0

    mid = (low + high) / 2

    count = 0

    count += mergeSort(low, mid)
    count += mergeSort(mid + 1, high)

    count += countPairs(low, mid, high)

    merge(low, mid, high)

    return count
```

Why count before merging?

Before merging:

- The left and right halves remain separately sorted.
- Their original division is known.
- We can ensure `i` belongs to the left half and `j` belongs to the right half.

Example

```
Left  = [1,3,5]
Right = [1,2,3]
```

For `1`:

```
1 > 2 × 1 → false
```

Count added:

```
0
```

For `3`:

```
3 > 2 × 1 → true
3 > 2 × 2 → false
```

Count added:

```
1
```

For `5`:

```
5 > 2 × 1 → true
5 > 2 × 2 → true
5 > 2 × 3 → false
```

Count added:

```
2
```

Total cross-pairs:

```
3
```

Important overflow issue

Do not write only:

```
arr[i] > 2 * arr[right]
```

If values are large, multiplication may overflow.

Use:

```
arr[i] > 2LL * arr[right]
```

or convert to a 64-bit integer first.

Complexity

- Merge sort levels: `O(log n)`
- Work per level: `O(n)`
- Total time: `O(n log n)`
- Space: `O(n)`

Revision key

> Sort both halves, count valid cross-pairs using a forward-only pointer, then merge.

Arrays Part III — Final Revision Table

Problem	Optimal technique	Time	Extra space
Search in 2D Matrix	Binary search on virtual 1D array	`O(log(mn))`	`O(1)`
Pow(x,n)	Binary exponentiation	`O(log |n|)`	`O(1)`
Majority Element `> n/2`	Moore’s Voting	`O(n)`	`O(1)`
Majority Element `> n/3`	Extended Moore’s Voting	`O(n)`	`O(1)`
Grid Unique Paths	Combinations	`O(min(m,n))`	`O(1)`
Reverse Pairs	Modified merge sort	`O(n log n)`	`O(n)`

One-Line Memory Tricks

```
Search Matrix:
Treat the matrix as a sorted 1D array.

Pow(x,n):
Odd power multiplies answer; every step squares the base.

Majority > n/2:
One candidate survives pairwise cancellation.

Majority > n/3:
At most two answers, so maintain two candidates.

Unique Paths:
Arrange the fixed number of right and downward moves.

Reverse Pairs:
Count arr[i] > 2 × arr[j] before merging sorted halves.
```

Pattern Recognition

```
Globally sorted matrix
→ Virtual binary search

Large exponent
→ Binary exponentiation

Element occurring more than half
→ Moore’s Voting with one candidate

Element occurring more than one-third
→ Moore’s Voting with two candidates

Fixed directional moves in a grid
→ Combinations

Count special ordered pairs
→ Modified merge sort
```

Most Important Interview Mistakes

Search in Matrix

```
Use number of columns for:

row = mid / columns
column = mid % columns
```

Do not divide by the number of rows.

Pow(x,n)

```
Convert n to a 64-bit integer before negating it.
```

This prevents overflow for `INT_MIN`.

Majority Element

```
Verify the candidate if existence is not guaranteed.
```

Majority Element II

```
The two candidates must be different.
Always verify both after candidate selection.
```

Unique Paths

```
Use 64-bit or larger numeric types when path counts can be large.
```

Reverse Pairs

```
Use 2LL × arr[right] to prevent integer overflow.
Count pairs before merging the halves.
```